

أثر (Ather) — Full Project Documentation

Table of Contents

1. [Project Overview](#)
 2. [System Architecture](#)
 3. [Frontend Architecture](#)
 4. [Backend Architecture](#)
 5. [Database Design](#)
 6. [API Reference](#)
 7. [Authentication & Authorization](#)
 8. [Real-time Features \(SignalR\)](#)
 9. [AI Integrations](#)
 10. [Background Services](#)
 11. [User Roles & Permissions](#)
 12. [Pages & Routes](#)
 13. [Component Library](#)
 14. [Deployment](#)
 15. [Environment Configuration](#)
-

1. Project Overview

Ather (أثر, meaning "Monument" in Arabic) is a full-stack Egyptian tourism platform that connects tourists with verified local tour guides. It leverages AI for trip planning and monument recognition, provides real-time communication, and includes comprehensive admin tools.

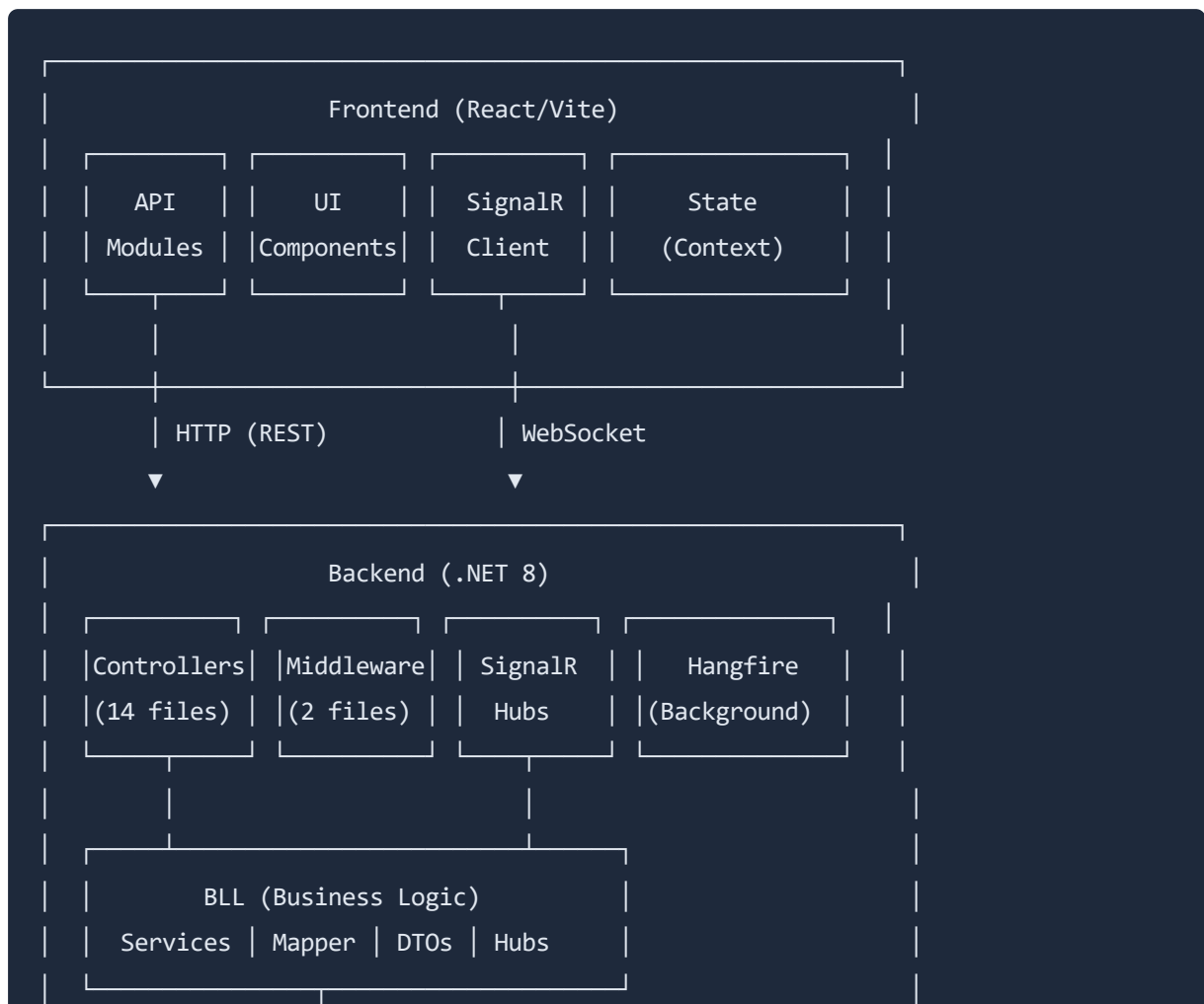
Target Users:

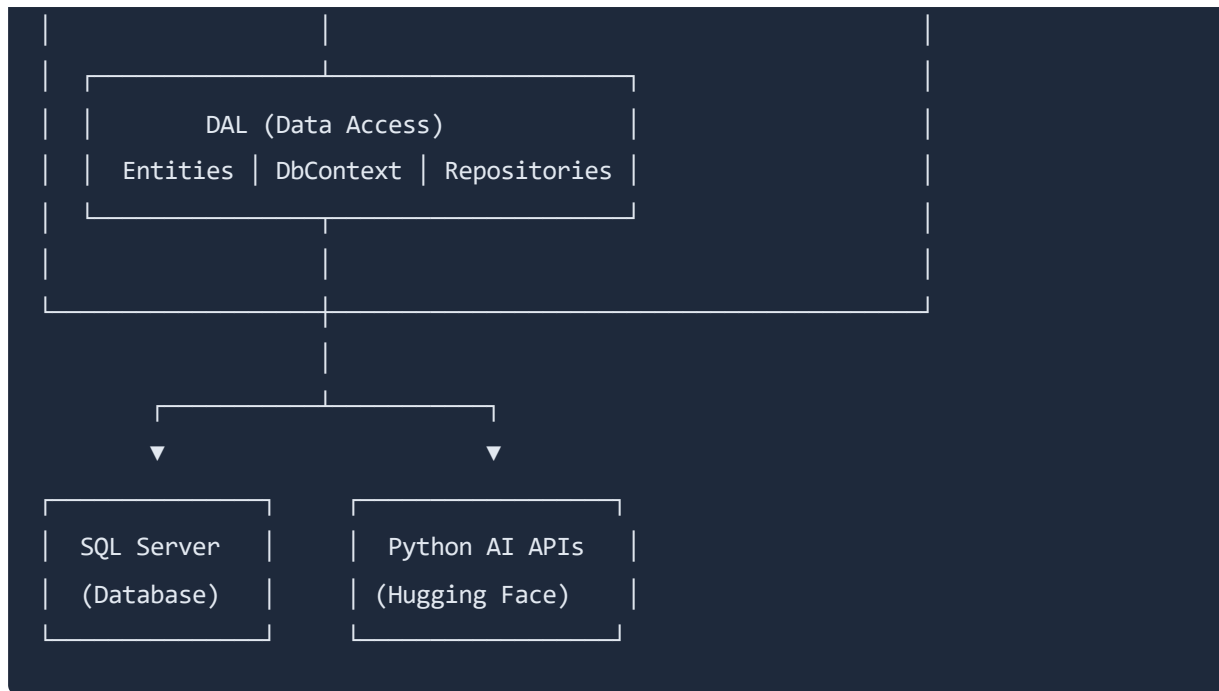
- **Tourists** — Local and international visitors to Egypt
- **Tour Guides** — Verified professionals offering guided tours
- **Admins** — Platform moderators and managers

Key Differentiators:

- AI-powered trip planning (budget, duration, interests)
- AI monument recognition (photo → historical info in 6 languages)
- Real-time chat between tourists and guides
- Instagram-like stories for travel content
- Automatic commission and debt management

2. System Architecture





Layer Responsibilities

Layer	Project	Responsibility
Presentation	FinalProject	Controllers, middleware, DI setup, configuration
Business	BLL	Service implementations, DTOs, SignalR hubs, business rules
Data	DAL	Entities, DbContext, repositories, migrations

3. Frontend Architecture

3.1 Tech Stack Details

Library	Version	Purpose
React	18.3.1	UI framework
TypeScript	5.5.3	Type safety

Library	Version	Purpose
Vite	5.3.3	Build tool & HMR
Tailwind CSS	4.0.0-alpha.14	Utility-first CSS
React Router	6.24.1	Client-side routing
Axios	1.7.2	HTTP client
@microsoft/signalr	8.0.7	Real-time WebSocket
Recharts	3.8.1	Admin charts
Lucide React	0.400.0	Icons

3.2 Directory Structure

```

src/
├── api/                                # 14 API modules
│   ├── axios.ts                       # Axios instance + envelope interceptor
│   ├── auth.ts                        # Authentication endpoints
│   ├── admin.ts                       # Admin endpoints
│   ├── places.ts                     # Place CRUD + user photos
│   ├── guides.ts                     # Guide profile/wallet
│   ├── bookings.ts                   # Booking CRUD + itinerary
│   ├── plans.ts                      # AI + manual plans
│   ├── stories.ts                    # Story CRUD + interactions
│   ├── reviews.ts                   # Reviews CRUD
│   ├── chat.ts                       # Chat history + send
│   ├── notifications.ts              # Notification CRUD
│   ├── tourists.ts                   # Tourist profile/plans
│   ├── payments.ts                   # Payment processing
│   └── aiGuide.ts                    # AI monument analysis
├── components/
│   ├── ai/AiGuideBot.tsx             # Floating AI chatbot
│   ├── brand/Logo.tsx                # Logo component
│   ├── layout/                       # 6 layout components
│   └── shared/                       # 4 card components

```

```

|   ├── stories/           # StoryStrip component
|   └── ui/                # 16 reusable UI components
|
|   ├── context/           # 4 React contexts
|   │   ├── AuthContext.tsx # Auth state + JWT management
|   │   ├── ThemeContext.tsx # Light/dark mode
|   │   ├── ToastContext.tsx # Toast notifications
|   │   └── NotificationContext.tsx # Real-time notification state
|
|   ├── pages/             # 30 page components
|   │   ├── auth/          # 4 pages
|   │   ├── shared/        # 10 pages
|   │   ├── tourist/       # 6 pages
|   │   ├── guide/         # 4 pages
|   │   └── admin/         # 7 pages
|
|   ├── routes/index.tsx   # Route definitions
|   ├── services/signalR.ts # SignalR connection + handlers
|   ├── types/index.ts     # TypeScript interfaces (429 lines)
|   └── utils/             # 6 utility modules

```

3.3 State Management

The app uses **React Context** (no Redux):

Context	State	Purpose
AuthContext	user , token , isAuthenticated , hasRole()	Auth state across app
ThemeContext	theme , toggleTheme()	Dark/light mode
ToastContext	toasts , showToast() , hideToast()	Global toast notifications
NotificationContext	unreadCount , pendingBookingsCount , unreadChatCount , notifications	SignalR-driven state

3.4 Axios Interceptor Pattern

The API layer uses a sophisticated interception pattern:

Request Interceptor: Attaches JWT token from localStorage to every request.

Response Interceptor: Unwraps the standard API envelope:

```
// Backend envelope shape
{
  isSuccess: boolean
  message?: string
  data?: T
  [key: string]: unknown // Extra fields (bookingId, planId, etc.)
}
```

On 401 responses, it auto-clears auth and redirects to `/login` (only for previously authenticated requests).

3.5 Routing Structure

The app uses a hierarchical route structure:

```
<RouterProvider> (creates browser router)
├─ / → MainLayout
|   ├─ / → LandingPage (public)
|   ├─ /explore → ExplorePage (public)
|   │   ├─ /explore/:id → PlaceDetailPage (public)
|   │   └─ /guides → GuidesPage (public)
|   │       ├─ /guides/:id → GuideDetailPage (public)
|   │       └─ /login → LoginPage (public)
|   │           └─ /register → RegisterPage (public)
|   │               └─ /verify-otp → VerifyOtpPage (public)
|   │                   └─ /forget-password → ForgetPasswordPage (public)
|   └─ ProtectedRoute (any auth)
|       ├─ /stories → StoriesPage
|       ├─ /profile → ProfilePage
|       └─ /notifications → NotificationsPage
|           └─ /chat → ChatPage
```

```

| | | └─ ProtectedRoute (Tourist)
| | |   └─ /plans/create → CreatePlanPage
| | |   └─ /plans        → MyPlansPage
| | |   └─ /plans/:id    → PlanDetailPage
| | |   └─ /bookings     → MyBookingsPage
| | |   └─ /bookings/create → CreateBookingPage
| | |   └─ /bookings/:id → BookingDetailPage
| | |
| | └─ ProtectedRoute (Guide)
| |   └─ /guide/profile → GuideProfilePage
| |   └─ /guide/bookings → GuideBookingsPage
| |   └─ /guide/bookings/:id → BookingDetailPage
| |   └─ /guide/wallet → GuideWalletPage
| |   └─ /guide/stories → GuideStoriesPage
| |
| └─ /admin → AdminLayout
|   └─ /admin → AdminDashboardPage
|   └─ /admin/users → AdminUsersPage
|   └─ /admin/pending-guides → AdminPendingGuidesPage
|   └─ /admin/places → AdminPlacesPage
|   └─ /admin/reviews → AdminReviewsPage
|   └─ /admin/stories → AdminStoriesPage
|   └─ /admin/notifications → AdminNotificationsPage
|
└─ * → Redirect to /

```

4. Backend Architecture

4.1 Project Structure

```

Backend/
└─ FinalProjectAPI.sln
|
└─ FinalProject/           # Web API (entry point)
  └─ Program.cs            # DI, middleware, configuration

```

```

|   ├── Controllers/           # 13 controllers
|   ├── MiddleWare/           # Exception + BanCheck middleware
|   ├── Common/               # Extension methods
|   ├── Properties/           # launchSettings + publish profiles
|   └── FinalProject.csproj
|
|   ├── BLL/                  # Business Logic Layer
|   |   ├── Service/          # 22 service implementations
|   |   |   ├── Implementation/ # Concrete services
|   |   |   └── Abstraction/    # Service interfaces
|   |   ├── ModelVm/          # DTOs / ViewModels
|   |   ├── Hubs/              # SignalR hubs
|   |   ├── Mapper/            # AutoMapper profile
|   |   ├── Helper/            # Upload + OTP utilities
|   |   ├── Global/            # Global usings
|   |   ├── Common/            # Common classes
|   |   └── BLL.csproj
|   |
|   └── DAL/                  # Data Access Layer
|       ├── DataApp/          # DbContext
|       ├── Entity/            # 17 entity classes
|       ├── Repo/              # Repository pattern
|       ├── Enum/              # Enumerations
|       ├── Errors/            # Error models
|       ├── Migrations/        # EF Core migrations
|       ├── Global/            # Global usings
|       └── DAL.csproj

```

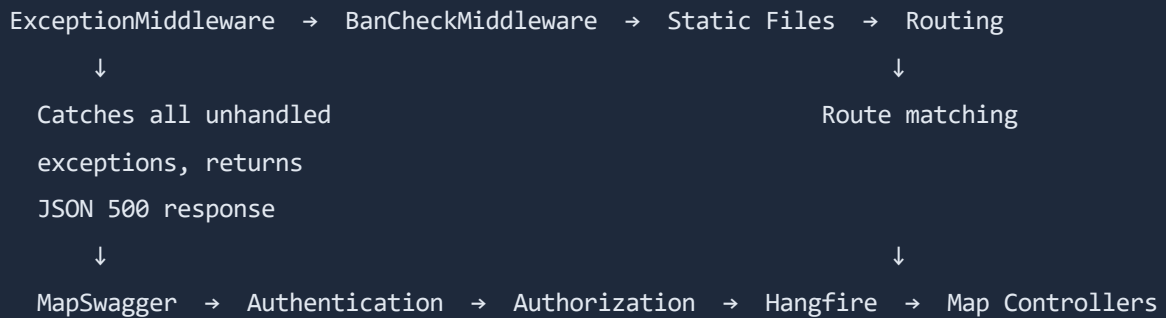
4.2 Dependency Injection (Program.cs)

The `Program.cs` registers services in this order:

1. **Database** — `AppDbContext` with SQL Server
2. **Identity** — ASP.NET Core Identity with JWT Bearer
3. **Repositories** — Scoped repository implementations
4. **Services** — Scoped service implementations
5. **Hangfire** — Background job server with SQL Server storage
6. **SignalR** — Real-time hubs (`/chathub` , `/notificationhub`)

7. **CORS** — Permissive policy for development
8. **Swagger** — API documentation
9. **Middleware** — Exception → BanCheck → Static Files → Routing → Auth → Hangfire
Dashboard → Map Controllers → Map Hubs

4.3 Middleware Pipeline



ExceptionMiddleware — Global error handler that catches all exceptions and returns a consistent JSON envelope with status 500.

BanCheckMiddleware — On every authenticated request, checks if the user's `LockoutEnd` is set and in the future, returning 403 with Arabic message if banned.

4.4 Services Layer (22 services)

Service	Responsibility
AuthService	Registration, login, email verification, password reset, OTP
AdminService	Dashboard stats, user management, guide approval
PlaceService	Place CRUD, filtering, user photo management
BookingService	Booking request/accept/reject/complete, itinerary
GuideService	Guide profile, wallet, dues payment
TouristService	Tourist profile, AI/manual plans
PlanService	AI plan generation, manual plan creation
ReviewService	Review CRUD (3 types)

Service	Responsibility
StoryService	Story creation, expiry, view/love interactions
ChatService	Chat history, send message
PaymentService	Payment processing, wallet updates
EmailService	Email sending via MailKit
FileService	File upload handling
AIRecommendation	Place recommendations via Python API
AiIntegrationService	AI trip plan generation via Python API
AiIntgrationModelChat	Monument analysis via Python API
InteractionService	User-place interaction tracking
AppNotificationService	Notification CRUD, SignalR broadcast
ExternalPlaceApiService	External place data API integration
BookingUpdateWorker	Background: auto-complete bookings
DebtMonitorWorker	Background: monitor guide debt
ChatService	Chat message persistence

5. Database Design

5.1 Entity Relationship Diagram

AppUser (AspNetUsers)

```

|— Tourist (1:1) ——— AiPlan (1:N)
|
|               |— ManualPlan (1:N) — ManualPlanItem (1:N)
|               |— Booking (as TouristId) (1:N)
|               |— Review (as TouristId) (1:N)

```

```

└── UserPlaceInteraction (1:N)
└── TourGuide (1:1) └── Booking (as GuideId) (1:N)
                    ├── Review (as GuideId) (1:N)
                    ├── Payment (1:N)
                    └── Story (1:N)
└── AppNotification (1:N)
└── RefreshToken (1:N)
└── Story (as UserId) (1:N)
└── ChatMessage (as SenderId/ReceiverId) (1:N)
└── PlacePhoto (as UserId) (1:N)

Place └── PlacePhoto (1:N)
      ├── Review (as PlaceId) (1:N)
      ├── ManualPlanItem (1:N)
      ├── UserPlaceInteraction (1:N)
      └── Booking (indirect via plans)

Booking └── ChatMessage (1:N)
        └── Review (1:N via bookingId)

Story └── StoryInteraction (1:N)

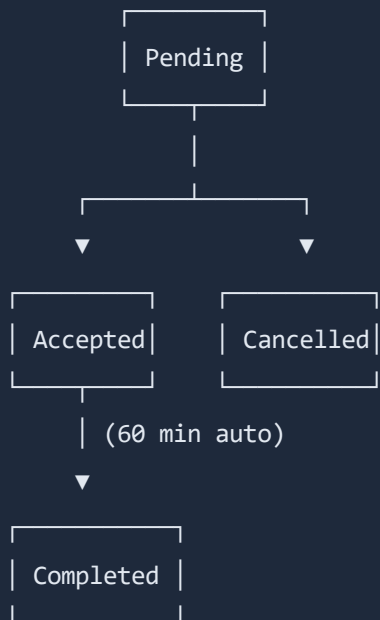
```

5.2 Entity Details

Entity	Key Fields	Notes
AppUser	Id, UserName, Email, Age, Country, Gender, UrlProfile	Extends IdentityUser
Tourist	Id, AppUserId, Name, Language, Rate	1:1 with AppUser
TourGuide	Id, AppUserId, NationalIdImage, Bio, Languages, PriceOfDay, WalletBalance,	1:1 with AppUser, has cancellation strikes

Entity	Key Fields	Notes
	OutstandingBalance, IsSuspended, DebtStartDate	
Place	Id, Name, Type, Lat, Lng, City, Description, TicketPrice, MainImageUrl, AffiliateLink, IdFromModel	Supports AI model ID linking
Booking	Id, TouristId, GuidId, AiPlanId, ManualPlanId, TotalPrice, Commission%, State	State machine: Pending→Accepted→Completed or Cancelled
AiPlan	Id, Name, Description, CountDay, Budget, Type, TouristId, AiResponseJson	AI-generated JSON response
ManualPlan	Id, Name, StartDate, TouristId	User-created plan
Review	Id, Rate, Comment, BookingId, TouristId, GuidId, PlaceId, ReviewType	3 review types via enum
Story	Id, UserId, City, Description, MediaUrl, MediaType, ExpiresAt	24h expiry
ChatMessage	Id, BookingId, SenderId, ReceiverId, Content, SentAt, IsRead	Per-booking messages
Payment	Id, GuidId, AmountPaid, OutBalance, PayMethod	Guide debt payments
AppNotification	Id, Title, Message, Target (All/Tourist/Guide), IsRead	Broadcast notifications

5.3 Booking State Machine



6. API Reference

6.1 Authentication Endpoints

POST /api/Account/login

```

// Request
{ "email": "string", "password": "string" }

// Response (200)
{
  "isSuccess": true,
  "data": {
    "userId": "string",
    "userName": "string",
    "email": "string",
    "role": "Tourist|Guide|Admin",
    "profileImageUrl": "string|null",
    "token": "string (JWT)"
  }
}
  
```

```
}
}
```

POST /api/Account/register-tourist

```
// Request (FormData)
{
  "userName": "string",
  "email": "string",
  "password": "string",
  "age": "number",
  "country": "string",
  "gender": "Male|Female"
}
```

POST /api/Account/register-guide

```
// Request (FormData)
{
  "userName": "string",
  "email": "string",
  "password": "string",
  "age": "number",
  "country": "string",
  "gender": "Male|Female",
  "nationalIdImage": "File (image)",
  "bio": "string",
  "languages": "string (comma-separated)",
  "priceOfDay": "number"
}
```

6.2 Admin Endpoints

GET /api/Admin/dashboard-stats — Returns comprehensive dashboard statistics:

- Total users, guides, tourists, places, bookings, revenue
- Monthly booking/revenue data for charts
- Booking status distribution

- Top guides by rating
- Tourist demographics (age groups, gender, country, language)

GET /api/Admin/pending-guides — Lists guides awaiting approval with their national ID images.

6.3 AI Endpoints

POST /api/Plans/generate — Generates an AI trip plan:

```
// Request
{
  "budget": "number",
  "countDay": "number",
  "type": "Cultural|Adventure|Relaxation|Historical|Religious",
  "governorate": "string (e.g., Cairo, Luxor, Aswan)"
}

// The backend calls an external Python AI API and stores the response
```

POST /api/AiChatModel?lang=ar — Analyzes monument image:

- Accepts: `multipart/form-data` with `file` field
- Returns: AI-generated monument description in requested language

6.4 SignalR Hubs

ChatHub (`/chathub`):

- `JoinBookingRoom(bookingId)` — Join a booking's chat room
- `LeaveBookingRoom(bookingId)` — Leave room
- `SendMessage(bookingId, message)` — Send message (broadcast to room)
- `ReceiveMessage(message)` — Client event for incoming messages

NotificationHub (`/notificationhub`):

- Users auto-join: `user:{userId}` (personal), role group (`Tourists` / `Guides` / `Admins`)
- Booking request events: tourist requests, guide cancels
- Booking state changes: accepted, rejected, completed, cancelled

- New message alerts with unread count
- Admin broadcast notifications

7. Authentication & Authorization

7.1 Flow Diagram

User → Register (FormData) → Email OTP → Verify OTP → Login → JWT Token

Frontend stores in localStorage
(ather_token, ather_user)

Axios Interceptor attaches
Authorization: Bearer <token>

Backend validates JWT
BanCheckMiddleware checks ban
[Authorize] + [Authorize(Roles)]

Controller processes request

7.2 JWT Token Structure

```
{
  "sub": "userId (guid)",
  "email": "user@example.com",
  "username": "User Name",
  "role": "Tourist|Guide|Admin",
  "profileImageUrl": "...",
  "exp": 1719000000,
```



```
"iss": "FinalProject",
"aud": "FinalProjectUsers"
}
```

7.3 Role Hierarchy

- **Admin** — Access to all admin routes and APIs
- **Guide** — Access to guide-specific routes and APIs
- **Tourist** — Access to tourist-specific routes and APIs
- **Public** — Access to browse places, guides, and auth pages only

8. Real-time Features (SignalR)

8.1 Connection Lifecycle

```
Frontend connects → /notificationhub
  → Server adds to group: user:{userId}
  → Server adds to role group: (Tourists|Guides|Admins)

Frontend connects → /chathub
  → Joins specific rooms: JoinBookingRoom(bookingId)
  → Receives messages: ReceiveMessage
```

8.2 Event Types

Event	Source	Trigger
ReceiveNotification	Admin	Admin sends broadcast
BookingRequested	Tourist	Tourist creates booking → notifies guide
BookingStateChanged	Server	Accept/Reject/Complete/Cancel → notifies both parties

Event	Source	Trigger
NewMessage	Chat	New chat message → notifies receiver
UnreadCountUpdate	Server	Updates unread badge counts

9. AI Integrations

9.1 Trip Plan Generator

- **Endpoint:** External Python API on Hugging Face Spaces
- **Input:** Budget, days, type (Cultural/Adventure/etc.), governorate
- **Output:** JSON itinerary with day-by-day schedule including places, activities, costs
- **Storage:** Saved as `AiPlan.AiResponseJson` in database

9.2 Monument Guide

- **Input:** User-uploaded image/audio + language code
- **Output:** Text description + optional audio URL of monument history
- **Languages Supported:** Arabic, English, French, German, Italian, Spanish
- **Frontend:** Floating AI Guide Bot accessible from any page

9.3 Place Recommendations

- **Input:** User interactions (views, favorites, trips)
- **Output:** Personalized place recommendations
- **Trigger:** Called from `TouristsController` for logged-in tourists

10. Background Services

BookingUpdateWorker

- Runs every 60 seconds (configurable via Hangfire recurring job)
- Finds bookings with:
 - `State == Accepted`
 - `UpdatedAt < DateTime.UtcNow - 60 minutes`
- Automatically completes them
- Applies guide commission debt (5% of total price) to guide's outstanding balance

DebtMonitorWorker

- Runs daily via Hangfire recurring job
- Finds guides with `OutstandingBalance > 0` and `DebtStartDate > 30 days`
- Sets `IsSuspended = true` for those guides
- Sends warning emails via MailKit
- Admin dashboard shows debt status

11. User Roles & Permissions

Feature	Public	Tourist	Guide	Admin
Browse Places	✓	✓	✓	✓
View Guides	✓	✓	✓	✓
Login/Register	✓	-	-	-
Create AI Plan	✗	✓	✗	✗
Create Manual Plan	✗	✓	✗	✗
Book Guide	✗	✓	✗	✗
Manage Bookings	✗	✓ (own)	✓ (assigned)	✗
Chat	✗	✓ (per booking)	✓ (per booking)	✗
Post Stories	✗	✓	✓	✗

Feature	Public	Tourist	Guide	Admin
Reviews	X	✓	✓	X
Guide Wallet	X	X	✓	✓
Admin Dashboard	X	X	X	✓
User Management	X	X	X	✓
Guide Approval	X	X	X	✓
Content Moderation	X	X	X	✓
Place CRUD	X	X	X	✓
Broadcast Notifications	X	X	X	✓

12. Pages & Routes

Public Pages

Route	Page	Description
/	Landing	Hero video, governorate cards, features, recommended places, story strip
/explore	Explore	Browse all monuments with type filter
/explore/:id	Place Detail	Place info, reviews, photo upload
/guides	Guides	List verified guides
/guides/:id	Guide Detail	Guide profile, reviews, booking button
/login	Login	Email + password login
/register	Register	Tourist or guide registration
/verify-otp	Verify OTP	Email verification

Route	Page	Description
/forget-password	Forget Password	Password reset flow

Tourist Pages

Route	Page	Description
/plans/create	Create Plan	AI or manual trip planning
/plans	My Plans	List AI + manual plans (tabs)
/plans/:id	Plan Detail	Itinerary view
/bookings	My Bookings	Booking list with status
/bookings/create	Create Booking	Select plan + guide
/bookings/:id	Booking Detail	Full booking info

Guide Pages

Route	Page	Description
/guide/profile	Profile	Edit guide profile
/guide/bookings	Bookings	Accept/reject/complete bookings
/guide/wallet	Wallet	Balance, dues, pay
/guide/stories	Stories	Create/manage stories

Admin Pages

Route	Page	Description
/admin	Dashboard	Stats, charts, revenue, demographics

Route	Page	Description
<code>/admin/users</code>	Users	Ban/delete users
<code>/admin/pending-guides</code>	Pending Guides	Approve/reject guides
<code>/admin/places</code>	Places	CRUD places
<code>/admin/reviews</code>	Reviews	Moderate reviews
<code>/admin/stories</code>	Stories	Moderate stories
<code>/admin/notifications</code>	Broadcast	Send notifications

13. Component Library

UI Components (16)

Component	Props	Description
Button	<code>variant</code> , <code>size</code> , <code>loading</code> , <code>disabled</code>	Styled button with loading state
Input	<code>label</code> , <code>error</code> , <code>icon</code>	Form input with validation
Select	<code>options</code> , <code>value</code> , <code>onChange</code>	Dropdown select
Textarea	<code>label</code> , <code>error</code> , <code>rows</code>	Multi-line text input
Card	<code>children</code> , <code>className</code>	Container card
Modal	<code>open</code> , <code>onClose</code> , <code>title</code>	Modal dialog
Tabs	<code>tabs: {label, content}[]</code>	Tabbed interface
Badge	<code>variant</code> , <code>children</code>	Status badge
Avatar	<code>src</code> , <code>alt</code> , <code>size</code>	User avatar

Component	Props	Description
StarRating	rating , onChange , readOnly	Star rating display/input
Skeleton	className	Loading skeleton
EmptyState	icon , title , description	Empty state placeholder
ThemeToggle	-	Dark/light mode toggle
ConfirmDialog	open , title , message , onConfirm , onCancel	Confirmation dialog
LanguageMultiSelect	value , onChange	Multi-language selector

14. Deployment

Backend Deployment (runasp.net)

- Hosted at: `https://ather.runasp.net`
- Deployed via WebDeploy (publish profile included)
- SQL Server database connection configured in `appsettings.json`

Frontend Build

```
cd Frontend
npm run build    # Output: dist/
npm run preview  # Preview production build
```

Vite Proxy Configuration

In development mode (`vite.config.ts`):

```
/api/*           → https://ather.runasp.net/api/*  
/chathub/*       → https://ather.runasp.net/chathub/*  
/notificationhub/* → https://ather.runasp.net/notificationhub/*
```

15. Environment Configuration

Frontend (.env)

```
VITE_API_BASE_URL=http://localhost:5008 # API base URL
```

Backend (appsettings.json)

```
{  
  "ConnectionStrings": {  
    "conString": "Server=...;Database=AtherDB;..."  
  },  
  "JWT": {  
    "ValidIssuer": "FinalProject",  
    "ValidAudience": "FinalProjectUsers",  
    "Secret": "...",  
    "Duration": 7  
  },  
  "EmailSettings": {  
    "SmtpServer": "smtp.gmail.com",  
    "Port": 587,  
    "SenderEmail": "...",  
    "SenderPassword": "..."  
  }  
}
```


Documentation generated for Ather (أثر) — Egyptian Tourism Platform